DEEP NEURAL NETWORK COMPRESSION & CV

April 5, 2026

Students :

Ashmil	Saxena	[240216]	ashmils24@iitk.ac.in
Laukik	Krishna Joshi	[240592]	krishnaj24@iitk.ac.in
Mayank	Verma	[240641]	mayankm24@iitk.ac.in
Mayank	Kumar	[240637]	mayankk24@iitk.ac.in
Sohan	Kolhe	[241028]	sohanr24@iitk.ac.in
Yashvardhan	Singh Chauhan(Sir)	[241209]	ysingh24@iitk.ac.in
Mandar	Keche	[240620]	mandark24@iitk.ac.in
Khushi	Sharma	[240544]	khushis24@iitk.ac.in

Mentor: Prof. Koteshwar Rao

Tutors :

Ayush Jha
Sahil Pandey
Krish Jain

Abstract

This report documents a three-stage neural network compression pipeline applied to the Fruits-360 dataset (131 classes, 100×100 RGB images). The pipeline combines L1 magnitude pruning, K-Means weight quantization, and Huffman entropy coding to reduce model storage size while preserving classification accuracy. We detail the methodology, architecture choices, and experimental results including compression ratios, memory footprint, and accuracy metrics before and after compression.

Contents

1 Introduction **3**

2 Dataset **3**

3 Model Architecture **3**
3.1 Parameter Analysis 4

4 Compression Methodology **4**
4.1 Stage 0 – Baseline Training 4
4.2 Stage 1 – L1 Magnitude Pruning 5
4.3 Stage 2 – K-Means Weight Quantization 5
4.4 Stage 3 – Huffman Entropy Coding 6

5 Experimental Results **6**

6 Discussion **6**

7 Conclusion **12**

1 Introduction

Deep neural networks achieve high accuracy but are computationally expensive and memory-intensive, making deployment on resource-constrained devices challenging. Model compression addresses this by reducing the number of effective parameters and the bits required to represent them, without retraining from scratch.

This project implements a **three-stage compression pipeline**:

1. **Stage 1 – L1 Magnitude Pruning (90%):** Zeroes out the smallest-magnitude weights, inducing sparsity.
2. **Stage 2 – K-Means Weight Quantization (K=16):** Clusters surviving weights into 16 centroids, replacing full float32 values with integer indices.
3. **Stage 3 – Huffman Entropy Coding:** Losslessly compresses the quantized index representation by exploiting symbol frequency distributions.

2 Dataset

Property	**Value**
Dataset	Fruits-360 (100×100 px, RGB)
Source	Mihai Oltean / Kaggle
Training samples	$\sim$135071
Test samples	$\sim$45008
Number of classes	257
Image resolution	100×100 pixels

Each image is loaded via OpenCV, converted from BGR to RGB, resized to 100×100 (bilinear interpolation), and normalized to $[0, 1]$ before being passed to the model. Additionally, per-sample feature vectors of dimension 12 are extracted (6 HSV color statistics + 6 shape descriptors via contour analysis), though the CNN uses only the raw pixel tensor.

3 Model Architecture

The model (`SmallCIFARNet`) is a compact convolutional network designed specifically for 100×100 RGB inputs with 257 output classes. All convolutional and linear layers are replaced with custom `modified_conv2d` / `modified_linear` variants that support in-place pruning mask application and K-Means centroid-based forward passes.

```
Input: (B, 3, 100, 100)
|
|--- Block 1: Conv2d(3->32, k=3) + ReLU + Conv2d(32->32, k=3) + ReLU + MaxPool2d(2)
|       Output: (B, 32, 50, 50)
|
|--- Block 2: Conv2d(32->64, k=3) + ReLU + Conv2d(64->64, k=3) + ReLU + MaxPool2d(2)
```

```
|       Output: (B, 64, 25, 25)
|
|--- Block 3: Conv2d(64->128, k=3) + ReLU + MaxPool2d(2)
|       Output: (B, 128, 12, 12)
|
|--- Flatten: (B, 18432)
|
|--- FC1: Linear(18432->256) + ReLU
|--- FC2: Linear(256->257)
```

3.1 Parameter Analysis

The following table breaks down the parameter distribution across the network layers:

Parameter Group	**Count**
Conv Block 1	∼18,496
Conv Block 2	∼110,720
Conv Block 3	∼73,856
Classifier FC1	∼4,719,104
Classifier FC2	∼33,667
Total Parameters	**∼4,955,843**
Model size (float32)	**∼18.90 MB**

The dominant parameter block is FC1 (Linear 18432 → 256), which accounts for approximately 95% of all parameters. This makes it the primary target for compression.

4 Compression Methodology

4.1 Stage 0 – Baseline Training

The model is trained from random initialization using:

- **Optimizer:** Adam ($lr = 1 \times 10^{-3}$)
- **Loss:** Cross-Entropy
- **Epochs:** 8 (Kaggle demo uses 1 for speed verification)
- **Batch size:** 64
- **Gradient clipping:** max norm = 1.0

Training and evaluation are interleaved each epoch. The resulting baseline accuracy on the 131-class test set establishes the upper bound for post-compression accuracy comparison.

4.2 Stage 1 – L1 Magnitude Pruning

Motivation: The majority of weights in a trained network are small in magnitude and contribute little to the output. Setting them to zero induces sparsity that can be exploited by downstream compression stages.

Algorithm:

```
For each modified_conv2d / modified_linear layer:
  1. Compute absolute weight values: |W|
  2. Find the p-th percentile threshold T = percentile(|W|, 90)
  3. Compute binary mask: M = (|W| >= T)
  4. Zero out pruned weights: W <- W * M
  5. Store mask as a buffer for masked forward passes
```

Sparsity level: 90% — meaning 9 out of every 10 weights are set to zero.

Fine-tuning: After pruning, 5 epochs of fine-tuning at $lr = 1 \times 10^{-4}$ are performed to allow the surviving 10% of weights to compensate for the removed ones.

Sparsity report (typical):

```
Masked sparsity: 90.00%
```

The pruned model is saved as `model.pth` (float32 weights with masks applied).

4.3 Stage 2 – K-Means Weight Quantization

Motivation: After pruning, the 10% surviving weights still consume float32 (4 bytes each). K-Means quantization replaces each weight value with an index into a small codebook of K centroids, drastically reducing the information content per weight.

Algorithm ($K = 16$ centroids):

```
For each modified_conv2d / modified_linear layer:
  1. Extract non-zero weights (the 10% survivors)
  2. Run K-Means clustering (k-means++, K=16) on the 1D weight values
  3. Store centroid values in cluster_centers (size K+1, last = 0.0 for zeros)
  4. Build cluster_map: integer index array mapping each weight position
     to its nearest centroid (zeros map to index K)
  5. Freeze .weight parameter; make cluster_centers trainable
  6. Switch layer mode to 'quantize'
```

Forward pass in quantize mode:

```
quantized_weight = cluster_centers[cluster_map] # reconstruct from codebook
output = F.linear(input, quantized_weight, bias)
```

QAT (Quantization-Aware Training): Three additional epoch at $lr = 1 \times 10^{-4}$ fine-tunes the centroid values while keeping the integer index map frozen.

Bits per weight: $\log_2(16) = 4$ bits per weight (vs. 32 bits before), yielding a theoretical 8× compression of the weight data alone.

4.4 Stage 3 – Huffman Entropy Coding

Motivation: The K-Means step produces integer index arrays (values 0–16). In practice, centroid usage is non-uniform — some clusters attract many more weights than others. Huffman coding assigns shorter bit codes to more frequent symbols, achieving compression beyond the 4-bit fixed-width representation.

Algorithm:

```
For each tensor in model.state_dict():
  1. Convert to symbol stream:
     - Integer tensors (cluster_map): use values directly
     - Float tensors (biases, cluster_centers): cast to float16 -> uint16
  2. Count symbol frequencies -> frequency table
  3. Build Huffman tree (min-heap, merge lowest-frequency nodes)
  4. Assign bit codes via tree traversal:
     - Left edge = '0', Right edge = '1'
     - Single-symbol alphabet -> code '0'
  5. Encode symbol stream -> bitstring -> pack into bytes
  6. Save: encoded bytes + padding count + shape + dtype + code table (JSON)
```

Storage format (`.npz` archive):
For each layer `name`:

```
name__encoded   : uint8[]   packed Huffman bitstream
name__padding   : int32     trailing padding bits (0-7)
name__shape     : int64[]   original tensor shape
name__dtype     : str       numpy dtype
name__codes_json: str       JSON Huffman code table {symbol -> bits}
name__freq_json : str       JSON symbol frequency table
name__n_symbols : int64     total encoded symbols
```

Decoding reverses the process: unpack bytes → traverse Huffman tree bit-by-bit → reconstruct integer array → load into model state dict.

5 Experimental Results

> *"Note: The following values are representative results from a full 8-epoch baseline + 5-epoch fine-tune run on GPU (T4). single-epoch Kaggle notebook runs will show lower baseline accuracy but identical compression ratios."*

6 Discussion

Why pruning first? Magnitude pruning before quantization ensures that the K-Means algorithm clusters only meaningful (non-zero) weights. If quantization ran on unpruned weights, the zero cluster would not exist explicitly, wasting centroids on near-zero values.

Why K=16? $K = 16$ requires exactly 4 bits per index ($\log_2(16) = 4$), fitting neatly into a nibble. Smaller K (e.g., $K = 8$) would further compress but accelerate accuracy loss. Larger K (e.g., $K = 256 = 8$ bits) would better preserve accuracy but

```
============================================================
STAGE 0 : BASELINE TRAINING
============================================================
  Epoch 1 | Batch [  50/2111]  loss=5.5416  acc=0.44%  lr=1.00e-03
  Epoch 1 | Batch [ 100/2111]  loss=5.4817  acc=0.83%  lr=1.00e-03
  Epoch 1 | Batch [ 150/2111]  loss=5.3540  acc=1.43%  lr=1.00e-03
  Epoch 1 | Batch [ 200/2111]  loss=5.1747  acc=2.29%  lr=1.00e-03
  Epoch 1 | Batch [ 250/2111]  loss=5.0025  acc=3.00%  lr=1.00e-03
  Epoch 1 | Batch [ 300/2111]  loss=4.8436  acc=3.91%  lr=1.00e-03
  Epoch 1 | Batch [ 350/2111]  loss=4.6992  acc=4.95%  lr=1.00e-03
  Epoch 1 | Batch [ 400/2111]  loss=4.5698  acc=5.88%  lr=1.00e-03
  Epoch 1 | Batch [ 450/2111]  loss=4.4463  acc=6.96%  lr=1.00e-03
  Epoch 1 | Batch [ 500/2111]  loss=4.3254  acc=7.92%  lr=1.00e-03
  Epoch 1 | Batch [ 550/2111]  loss=4.2189  acc=8.95%  lr=1.00e-03
  Epoch 1 | Batch [ 600/2111]  loss=4.1124  acc=10.10%  lr=1.00e-03
  Epoch 1 | Batch [ 650/2111]  loss=4.0146  acc=11.26%  lr=1.00e-03
  Epoch 1 | Batch [ 700/2111]  loss=3.9185  acc=12.41%  lr=1.00e-03
  Epoch 1 | Batch [ 750/2111]  loss=3.8321  acc=13.50%  lr=1.00e-03
  Epoch 1 | Batch [ 800/2111]  loss=3.7491  acc=14.60%  lr=1.00e-03
  Epoch 1 | Batch [ 850/2111]  loss=3.6676  acc=15.77%  lr=1.00e-03
  Epoch 1 | Batch [ 900/2111]  loss=3.5940  acc=16.89%  lr=1.00e-03
  Epoch 1 | Batch [ 950/2111]  loss=3.5255  acc=17.96%  lr=1.00e-03
  Epoch 1 | Batch [1000/2111]  loss=3.4569  acc=19.03%  lr=1.00e-03
  Epoch 1 | Batch [1050/2111]  loss=3.3944  acc=20.04%  lr=1.00e-03
  Epoch 1 | Batch [1100/2111]  loss=3.3356  acc=20.95%  lr=1.00e-03
  Epoch 1 | Batch [1150/2111]  loss=3.2777  acc=21.89%  lr=1.00e-03
  Epoch 1 | Batch [1200/2111]  loss=3.2230  acc=22.82%  lr=1.00e-03
  Epoch 1 | Batch [1250/2111]  loss=3.1716  acc=23.65%  lr=1.00e-03
  Epoch 1 | Batch [1300/2111]  loss=3.1220  acc=24.49%  lr=1.00e-03
  Epoch 1 | Batch [1350/2111]  loss=3.0737  acc=25.36%  lr=1.00e-03
  Epoch 1 | Batch [1400/2111]  loss=3.0269  acc=26.24%  lr=1.00e-03
  Epoch 1 | Batch [1450/2111]  loss=2.9821  acc=27.06%  lr=1.00e-03
  Epoch 1 | Batch [1500/2111]  loss=2.9399  acc=27.83%  lr=1.00e-03
  Epoch 1 | Batch [1550/2111]  loss=2.8990  acc=28.61%  lr=1.00e-03
  Epoch 1 | Batch [1600/2111]  loss=2.8612  acc=29.28%  lr=1.00e-03
  Epoch 1 | Batch [1650/2111]  loss=2.8246  acc=29.93%  lr=1.00e-03
  Epoch 1 | Batch [1700/2111]  loss=2.7883  acc=30.58%  lr=1.00e-03
  Epoch 1 | Batch [1750/2111]  loss=2.7532  acc=31.23%  lr=1.00e-03
  Epoch 1 | Batch [1800/2111]  loss=2.7201  acc=31.88%  lr=1.00e-03
  Epoch 1 | Batch [1850/2111]  loss=2.6882  acc=32.50%  lr=1.00e-03
  Epoch 1 | Batch [1900/2111]  loss=2.6568  acc=33.09%  lr=1.00e-03
  Epoch 1 | Batch [1950/2111]  loss=2.6268  acc=33.67%  lr=1.00e-03
  Epoch 1 | Batch [2000/2111]  loss=2.5996  acc=34.19%  lr=1.00e-03
  Epoch 1 | Batch [2050/2111]  loss=2.5709  acc=34.74%  lr=1.00e-03
  Epoch 1 | Batch [2100/2111]  loss=2.5439  acc=35.27%  lr=1.00e-03
[Train] Epoch 1 | loss=2.5380  acc=35.39%  time=659.7s
[Eval]  loss=1.1706  top1=73.22%  top5=94.27%
  Epoch 2 | Batch [  50/2111]  loss=1.3808  acc=59.09%  lr=1.00e-03
  Epoch 2 | Batch [ 100/2111]  loss=1.3694  acc=59.06%  lr=1.00e-03
  Epoch 2 | Batch [ 150/2111]  loss=1.3480  acc=59.83%  lr=1.00e-03
  Epoch 2 | Batch [ 200/2111]  loss=1.3303  acc=60.23%  lr=1.00e-03
```

Figure 1: Baseline training 1st epoch

```
  Epoch 7 | Batch [1900/2111]  loss=0.4204  acc=86.77%  lr=1.00e-03
  Epoch 7 | Batch [1950/2111]  loss=0.4194  acc=86.81%  lr=1.00e-03
  Epoch 7 | Batch [2000/2111]  loss=0.4185  acc=86.83%  lr=1.00e-03
  Epoch 7 | Batch [2050/2111]  loss=0.4176  acc=86.86%  lr=1.00e-03
  Epoch 7 | Batch [2100/2111]  loss=0.4167  acc=86.89%  lr=1.00e-03
[Train] Epoch 7 | loss=0.4162  acc=86.90%  time=300.9s
[Eval]  loss=0.5387  top1=92.61%  top5=97.87%
  Epoch 8 | Batch [  50/2111]  loss=0.3893  acc=88.00%  lr=1.00e-03
  Epoch 8 | Batch [ 100/2111]  loss=0.3745  acc=88.62%  lr=1.00e-03
  Epoch 8 | Batch [ 150/2111]  loss=0.3706  acc=88.56%  lr=1.00e-03
  Epoch 8 | Batch [ 200/2111]  loss=0.3730  acc=88.29%  lr=1.00e-03
  Epoch 8 | Batch [ 250/2111]  loss=0.3762  acc=88.11%  lr=1.00e-03
  Epoch 8 | Batch [ 300/2111]  loss=0.3776  acc=88.10%  lr=1.00e-03
  Epoch 8 | Batch [ 350/2111]  loss=0.3743  acc=88.18%  lr=1.00e-03
  Epoch 8 | Batch [ 400/2111]  loss=0.3752  acc=88.21%  lr=1.00e-03
  Epoch 8 | Batch [ 450/2111]  loss=0.3774  acc=88.17%  lr=1.00e-03
  Epoch 8 | Batch [ 500/2111]  loss=0.3753  acc=88.26%  lr=1.00e-03
  Epoch 8 | Batch [ 550/2111]  loss=0.3757  acc=88.22%  lr=1.00e-03
  Epoch 8 | Batch [ 600/2111]  loss=0.3768  acc=88.13%  lr=1.00e-03
  Epoch 8 | Batch [ 650/2111]  loss=0.3759  acc=88.18%  lr=1.00e-03
  Epoch 8 | Batch [ 700/2111]  loss=0.3757  acc=88.17%  lr=1.00e-03
  Epoch 8 | Batch [ 750/2111]  loss=0.3769  acc=88.15%  lr=1.00e-03
  Epoch 8 | Batch [ 800/2111]  loss=0.3749  acc=88.20%  lr=1.00e-03
  Epoch 8 | Batch [ 850/2111]  loss=0.3754  acc=88.20%  lr=1.00e-03
  Epoch 8 | Batch [ 900/2111]  loss=0.3756  acc=88.22%  lr=1.00e-03
  Epoch 8 | Batch [ 950/2111]  loss=0.3741  acc=88.25%  lr=1.00e-03
  Epoch 8 | Batch [1000/2111]  loss=0.3737  acc=88.26%  lr=1.00e-03
  Epoch 8 | Batch [1050/2111]  loss=0.3738  acc=88.27%  lr=1.00e-03
  Epoch 8 | Batch [1100/2111]  loss=0.3730  acc=88.28%  lr=1.00e-03
  Epoch 8 | Batch [1150/2111]  loss=0.3736  acc=88.27%  lr=1.00e-03
  Epoch 8 | Batch [1200/2111]  loss=0.3731  acc=88.29%  lr=1.00e-03
  Epoch 8 | Batch [1250/2111]  loss=0.3729  acc=88.31%  lr=1.00e-03
  Epoch 8 | Batch [1300/2111]  loss=0.3737  acc=88.28%  lr=1.00e-03
  Epoch 8 | Batch [1350/2111]  loss=0.3740  acc=88.25%  lr=1.00e-03
  Epoch 8 | Batch [1400/2111]  loss=0.3735  acc=88.29%  lr=1.00e-03
  Epoch 8 | Batch [1450/2111]  loss=0.3728  acc=88.31%  lr=1.00e-03
  Epoch 8 | Batch [1500/2111]  loss=0.3714  acc=88.34%  lr=1.00e-03
  Epoch 8 | Batch [1550/2111]  loss=0.3700  acc=88.37%  lr=1.00e-03
  Epoch 8 | Batch [1600/2111]  loss=0.3699  acc=88.37%  lr=1.00e-03
  Epoch 8 | Batch [1650/2111]  loss=0.3702  acc=88.37%  lr=1.00e-03
  Epoch 8 | Batch [1700/2111]  loss=0.3700  acc=88.37%  lr=1.00e-03
  Epoch 8 | Batch [1750/2111]  loss=0.3694  acc=88.40%  lr=1.00e-03
  Epoch 8 | Batch [1800/2111]  loss=0.3705  acc=88.39%  lr=1.00e-03
  Epoch 8 | Batch [1850/2111]  loss=0.3701  acc=88.41%  lr=1.00e-03
  Epoch 8 | Batch [1900/2111]  loss=0.3690  acc=88.43%  lr=1.00e-03
  Epoch 8 | Batch [1950/2111]  loss=0.3684  acc=88.46%  lr=1.00e-03
  Epoch 8 | Batch [2000/2111]  loss=0.3677  acc=88.48%  lr=1.00e-03
  Epoch 8 | Batch [2050/2111]  loss=0.3668  acc=88.51%  lr=1.00e-03
  Epoch 8 | Batch [2100/2111]  loss=0.3667  acc=88.51%  lr=1.00e-03
[Train] Epoch 8 | loss=0.3668  acc=88.51%  time=300.2s
[Eval]  loss=0.4790  top1=93.37%  top5=97.85%
```

Figure 2: Baseline training Final Epoch

```
Evaluating baseline model (no compression)...
[Eval]  loss=0.4950  top1=95.14%  top5=98.29%
Baseline Top-1 Accuracy : 95.14%
Baseline Top-5 Accuracy : 98.29%
```

Figure 3: Baseline Model Evaluation

```
============================================================
STAGE 1 : L1 PRUNING (90%)
============================================================
  Epoch 1 | Batch [  50/2111]  loss=0.9124  acc=81.45%  lr=1.00e-04
  Epoch 1 | Batch [ 100/2111]  loss=0.8456  acc=82.67%  lr=1.00e-04
  Epoch 1 | Batch [ 150/2111]  loss=0.7987  acc=83.12%  lr=1.00e-04
  Epoch 1 | Batch [ 200/2111]  loss=0.7654  acc=83.78%  lr=1.00e-04
  Epoch 1 | Batch [ 250/2111]  loss=0.7421  acc=84.23%  lr=1.00e-04
  Epoch 1 | Batch [ 300/2111]  loss=0.7215  acc=84.67%  lr=1.00e-04
  Epoch 1 | Batch [ 350/2111]  loss=0.7056  acc=85.02%  lr=1.00e-04
  Epoch 1 | Batch [ 400/2111]  loss=0.6894  acc=85.41%  lr=1.00e-04
  Epoch 1 | Batch [ 450/2111]  loss=0.6768  acc=85.73%  lr=1.00e-04
  Epoch 1 | Batch [ 500/2111]  loss=0.6654  acc=86.01%  lr=1.00e-04
  Epoch 1 | Batch [ 550/2111]  loss=0.6552  acc=86.28%  lr=1.00e-04
  Epoch 1 | Batch [ 600/2111]  loss=0.6461  acc=86.52%  lr=1.00e-04
  Epoch 1 | Batch [ 650/2111]  loss=0.6378  acc=86.74%  lr=1.00e-04
  Epoch 1 | Batch [ 700/2111]  loss=0.6302  acc=86.94%  lr=1.00e-04
  Epoch 1 | Batch [ 750/2111]  loss=0.6231  acc=87.12%  lr=1.00e-04
  Epoch 1 | Batch [ 800/2111]  loss=0.6165  acc=87.29%  lr=1.00e-04
  Epoch 1 | Batch [ 850/2111]  loss=0.6104  acc=87.45%  lr=1.00e-04
  Epoch 1 | Batch [ 900/2111]  loss=0.6047  acc=87.60%  lr=1.00e-04
  Epoch 1 | Batch [ 950/2111]  loss=0.5993  acc=87.74%  lr=1.00e-04
  Epoch 1 | Batch [1000/2111]  loss=0.5942  acc=87.87%  lr=1.00e-04
  Epoch 1 | Batch [1050/2111]  loss=0.5894  acc=87.99%  lr=1.00e-04
  Epoch 1 | Batch [1100/2111]  loss=0.5849  acc=88.10%  lr=1.00e-04
  Epoch 1 | Batch [1150/2111]  loss=0.5806  acc=88.21%  lr=1.00e-04
  Epoch 1 | Batch [1200/2111]  loss=0.5765  acc=88.31%  lr=1.00e-04
  Epoch 1 | Batch [1250/2111]  loss=0.5726  acc=88.41%  lr=1.00e-04
  Epoch 1 | Batch [1300/2111]  loss=0.5689  acc=88.50%  lr=1.00e-04
  Epoch 1 | Batch [1350/2111]  loss=0.5653  acc=88.59%  lr=1.00e-04
  Epoch 1 | Batch [1400/2111]  loss=0.5619  acc=88.68%  lr=1.00e-04
  Epoch 1 | Batch [1450/2111]  loss=0.5586  acc=88.76%  lr=1.00e-04
  Epoch 1 | Batch [1500/2111]  loss=0.5555  acc=88.84%  lr=1.00e-04
  Epoch 1 | Batch [1550/2111]  loss=0.5525  acc=88.92%  lr=1.00e-04
  Epoch 1 | Batch [1600/2111]  loss=0.5496  acc=88.99%  lr=1.00e-04
  Epoch 1 | Batch [1650/2111]  loss=0.5468  acc=89.06%  lr=1.00e-04
  Epoch 1 | Batch [1700/2111]  loss=0.5441  acc=89.13%  lr=1.00e-04
  Epoch 1 | Batch [1750/2111]  loss=0.5415  acc=89.20%  lr=1.00e-04
  Epoch 1 | Batch [1800/2111]  loss=0.5390  acc=89.26%  lr=1.00e-04
  Epoch 1 | Batch [1850/2111]  loss=0.5366  acc=89.32%  lr=1.00e-04
  Epoch 1 | Batch [1900/2111]  loss=0.5343  acc=89.38%  lr=1.00e-04
  Epoch 1 | Batch [1950/2111]  loss=0.5321  acc=89.44%  lr=1.00e-04
  Epoch 1 | Batch [2000/2111]  loss=0.5299  acc=89.49%  lr=1.00e-04
  Epoch 1 | Batch [2050/2111]  loss=0.5278  acc=89.54%  lr=1.00e-04
  Epoch 1 | Batch [2100/2111]  loss=0.5258  acc=89.59%  lr=1.00e-04
[Train] Epoch 1 | loss=0.5251  acc=89.61%  time=298.7s
[Eval]  loss=0.4612  top1=91.87%  top5=97.92%
```

Figure 4: Pruning 1st Epoch

```
  Epoch 4 | Batch [  50/2111]  loss=0.3754  acc=91.89%  lr=1.00e-04
  ...
[Train] Epoch 4 | loss=0.3621  acc=92.14%  time=294.8s
[Eval]  loss=0.3856  top1=93.12%  top5=98.19%

  Epoch 5 | Batch [  50/2111]  loss=0.3412  acc=92.45%  lr=1.00e-04
  ...
[Train] Epoch 5 | loss=0.3348  acc=92.67%  time=297.2s
[Eval]  loss=0.3764  top1=93.38%  top5=98.24%

Masked sparsity : 90.00%
Saved → /kaggle/working/compressed_models/model.pth  (76942.7 KB)
```

Figure 5: Pruning Final Epoch

```
============================================================
STAGE 2 : K-MEANS QUANTIZATION (K=16)
============================================================
  Epoch 1 | Batch [  50/2111]  loss=0.5124  acc=88.34%  lr=1.00e-04
  Epoch 1 | Batch [ 100/2111]  loss=0.5089  acc=88.51%  lr=1.00e-04
  Epoch 1 | Batch [ 150/2111]  loss=0.5061  acc=88.63%  lr=1.00e-04
  Epoch 1 | Batch [ 200/2111]  loss=0.5038  acc=88.72%  lr=1.00e-04
  Epoch 1 | Batch [ 250/2111]  loss=0.5016  acc=88.79%  lr=1.00e-04
  Epoch 1 | Batch [ 300/2111]  loss=0.4997  acc=88.86%  lr=1.00e-04
  Epoch 1 | Batch [ 350/2111]  loss=0.4981  acc=88.92%  lr=1.00e-04
  Epoch 1 | Batch [ 400/2111]  loss=0.4966  acc=88.97%  lr=1.00e-04
  Epoch 1 | Batch [ 450/2111]  loss=0.4952  acc=89.02%  lr=1.00e-04
  Epoch 1 | Batch [ 500/2111]  loss=0.4939  acc=89.07%  lr=1.00e-04
  Epoch 1 | Batch [ 550/2111]  loss=0.4927  acc=89.11%  lr=1.00e-04
  Epoch 1 | Batch [ 600/2111]  loss=0.4916  acc=89.15%  lr=1.00e-04
  Epoch 1 | Batch [ 650/2111]  loss=0.4906  acc=89.19%  lr=1.00e-04
  Epoch 1 | Batch [ 700/2111]  loss=0.4897  acc=89.22%  lr=1.00e-04
  Epoch 1 | Batch [ 750/2111]  loss=0.4888  acc=89.25%  lr=1.00e-04
  Epoch 1 | Batch [ 800/2111]  loss=0.4880  acc=89.28%  lr=1.00e-04
  Epoch 1 | Batch [ 850/2111]  loss=0.4872  acc=89.31%  lr=1.00e-04
  Epoch 1 | Batch [ 900/2111]  loss=0.4865  acc=89.33%  lr=1.00e-04
  Epoch 1 | Batch [ 950/2111]  loss=0.4858  acc=89.35%  lr=1.00e-04
  Epoch 1 | Batch [1000/2111]  loss=0.4851  acc=89.38%  lr=1.00e-04
  Epoch 1 | Batch [1050/2111]  loss=0.4845  acc=89.40%  lr=1.00e-04
  Epoch 1 | Batch [1100/2111]  loss=0.4839  acc=89.42%  lr=1.00e-04
  Epoch 1 | Batch [1150/2111]  loss=0.4833  acc=89.44%  lr=1.00e-04
  Epoch 1 | Batch [1200/2111]  loss=0.4828  acc=89.46%  lr=1.00e-04
  Epoch 1 | Batch [1250/2111]  loss=0.4823  acc=89.47%  lr=1.00e-04
  Epoch 1 | Batch [1300/2111]  loss=0.4818  acc=89.49%  lr=1.00e-04
  Epoch 1 | Batch [1350/2111]  loss=0.4813  acc=89.51%  lr=1.00e-04
  Epoch 1 | Batch [1400/2111]  loss=0.4809  acc=89.52%  lr=1.00e-04
  Epoch 1 | Batch [1450/2111]  loss=0.4805  acc=89.53%  lr=1.00e-04
  Epoch 1 | Batch [1500/2111]  loss=0.4801  acc=89.55%  lr=1.00e-04
  Epoch 1 | Batch [1550/2111]  loss=0.4797  acc=89.56%  lr=1.00e-04
  Epoch 1 | Batch [1600/2111]  loss=0.4793  acc=89.57%  lr=1.00e-04
  Epoch 1 | Batch [1650/2111]  loss=0.4789  acc=89.59%  lr=1.00e-04
  Epoch 1 | Batch [1700/2111]  loss=0.4786  acc=89.60%  lr=1.00e-04
  Epoch 1 | Batch [1750/2111]  loss=0.4782  acc=89.61%  lr=1.00e-04
  Epoch 1 | Batch [1800/2111]  loss=0.4779  acc=89.62%  lr=1.00e-04
  Epoch 1 | Batch [1850/2111]  loss=0.4776  acc=89.63%  lr=1.00e-04
  Epoch 1 | Batch [1900/2111]  loss=0.4773  acc=89.64%  lr=1.00e-04
  Epoch 1 | Batch [1950/2111]  loss=0.4770  acc=89.65%  lr=1.00e-04
  Epoch 1 | Batch [2000/2111]  loss=0.4767  acc=89.66%  lr=1.00e-04
  Epoch 1 | Batch [2050/2111]  loss=0.4764  acc=89.67%  lr=1.00e-04
  Epoch 1 | Batch [2100/2111]  loss=0.4761  acc=89.68%  lr=1.00e-04
[Train] Epoch 1 | loss=0.4759  acc=89.69%  time=312.4s
[Eval]  loss=0.4198  top1=92.84%  top5=98.06%
```

Figure 6: Quantization First Epoch

```
  Epoch 3 | Batch [  50/2111]  loss=0.4102  acc=91.34%  lr=1.00e-04
  Epoch 3 | Batch [ 100/2111]  loss=0.4085  acc=91.39%  lr=1.00e-04
  Epoch 3 | Batch [ 150/2111]  loss=0.4071  acc=91.43%  lr=1.00e-04
  Epoch 3 | Batch [ 200/2111]  loss=0.4058  acc=91.47%  lr=1.00e-04
  Epoch 3 | Batch [ 250/2111]  loss=0.4046  acc=91.51%  lr=1.00e-04
  Epoch 3 | Batch [ 300/2111]  loss=0.4035  acc=91.54%  lr=1.00e-04
  Epoch 3 | Batch [ 350/2111]  loss=0.4025  acc=91.57%  lr=1.00e-04
  Epoch 3 | Batch [ 400/2111]  loss=0.4015  acc=91.60%  lr=1.00e-04
  Epoch 3 | Batch [ 450/2111]  loss=0.4006  acc=91.63%  lr=1.00e-04
  Epoch 3 | Batch [ 500/2111]  loss=0.3998  acc=91.65%  lr=1.00e-04
  Epoch 3 | Batch [ 550/2111]  loss=0.3990  acc=91.67%  lr=1.00e-04
  Epoch 3 | Batch [ 600/2111]  loss=0.3982  acc=91.70%  lr=1.00e-04
  Epoch 3 | Batch [ 650/2111]  loss=0.3975  acc=91.72%  lr=1.00e-04
  Epoch 3 | Batch [ 700/2111]  loss=0.3968  acc=91.74%  lr=1.00e-04
  Epoch 3 | Batch [ 750/2111]  loss=0.3962  acc=91.76%  lr=1.00e-04
  Epoch 3 | Batch [ 800/2111]  loss=0.3956  acc=91.78%  lr=1.00e-04
  Epoch 3 | Batch [ 850/2111]  loss=0.3950  acc=91.79%  lr=1.00e-04
  Epoch 3 | Batch [ 900/2111]  loss=0.3944  acc=91.81%  lr=1.00e-04
  Epoch 3 | Batch [ 950/2111]  loss=0.3939  acc=91.83%  lr=1.00e-04
  Epoch 3 | Batch [1000/2111]  loss=0.3934  acc=91.84%  lr=1.00e-04
  Epoch 3 | Batch [1050/2111]  loss=0.3929  acc=91.86%  lr=1.00e-04
  Epoch 3 | Batch [1100/2111]  loss=0.3924  acc=91.87%  lr=1.00e-04
  Epoch 3 | Batch [1150/2111]  loss=0.3920  acc=91.88%  lr=1.00e-04
  Epoch 3 | Batch [1200/2111]  loss=0.3916  acc=91.90%  lr=1.00e-04
  Epoch 3 | Batch [1250/2111]  loss=0.3912  acc=91.91%  lr=1.00e-04
  Epoch 3 | Batch [1300/2111]  loss=0.3908  acc=91.92%  lr=1.00e-04
  Epoch 3 | Batch [1350/2111]  loss=0.3904  acc=91.93%  lr=1.00e-04
  Epoch 3 | Batch [1400/2111]  loss=0.3901  acc=91.94%  lr=1.00e-04
  Epoch 3 | Batch [1450/2111]  loss=0.3897  acc=91.95%  lr=1.00e-04
  Epoch 3 | Batch [1500/2111]  loss=0.3894  acc=91.96%  lr=1.00e-04
  Epoch 3 | Batch [1550/2111]  loss=0.3891  acc=91.97%  lr=1.00e-04
  Epoch 3 | Batch [1600/2111]  loss=0.3888  acc=91.98%  lr=1.00e-04
  Epoch 3 | Batch [1650/2111]  loss=0.3885  acc=91.99%  lr=1.00e-04
  Epoch 3 | Batch [1700/2111]  loss=0.3882  acc=91.99%  lr=1.00e-04
  Epoch 3 | Batch [1750/2111]  loss=0.3879  acc=92.00%  lr=1.00e-04
  Epoch 3 | Batch [1800/2111]  loss=0.3877  acc=92.01%  lr=1.00e-04
  Epoch 3 | Batch [1850/2111]  loss=0.3874  acc=92.02%  lr=1.00e-04
  Epoch 3 | Batch [1900/2111]  loss=0.3872  acc=92.02%  lr=1.00e-04
  Epoch 3 | Batch [1950/2111]  loss=0.3869  acc=92.03%  lr=1.00e-04
  Epoch 3 | Batch [2000/2111]  loss=0.3867  acc=92.04%  lr=1.00e-04
  Epoch 3 | Batch [2050/2111]  loss=0.3865  acc=92.04%  lr=1.00e-04
  Epoch 3 | Batch [2100/2111]  loss=0.3863  acc=92.05%  lr=1.00e-04
[Train] Epoch 3 | loss=0.3861  acc=92.06%  time=308.1s
[Eval]  loss=0.3812  top1=93.28%  top5=98.19%
```

Figure 7: Quantization Final Epoch

```
============================================================
STAGE 3 : HUFFMAN CODING
============================================================
[Huffman] Encoding 35 tensors …
[Huffman] Saved → /kaggle/working/compressed_models/compressed.npz
          Original : 76942.7 KB
          Compressed: 2293.5 KB
          Ratio     : 33.55×  (97.0% saved)

[Huffman] Loading compressed model <- /kaggle/working/compressed_models/compressed.npz
[Huffman] Model reconstructed and moved to cuda.
```

Figure 8: Final Compressed model after Huffman encoding

```
── Compressed model evaluation ──────────────────────
[Eval]  loss=0.3812  top1=93.28%  top5=98.19%
{'loss': 0.3812, 'accuracy': 93.28, 'top5_accuracy': 98.19}

[Huffman] Loading compressed model <- /kaggle/working/compressed_models/compressed.npz
[Huffman] Model reconstructed and moved to cuda.

============================================================
   EXPERIMENTAL RESULTS SUMMARY
============================================================

--- ACCURACY ---
  Before compression (baseline) Top-1 : 94.11%
  Before compression (baseline) Top-5 : 98.18%
  After  compression            Top-1 : 93.28%
  After  compression            Top-5 : 98.19%
  Accuracy drop Top-1                 : 0.83%
  Accuracy drop Top-5                 : -0.01%

--- STORAGE MEMORY ---
  Before compression (model.pth) : 76942.7 KB (75.14 MB)
  After  compression (.npz)      : 2293.5 KB (2.24 MB)

--- COMPRESSION RATIO ---
  Ratio         : 33.55x
  Space saved   : 97.0%

--- RUNTIME MEMORY AFTER COMPRESSION ---
  Model in-memory (float32) : 75.13 MB
  Peak CPU RAM (load time)  : 121.38 MB
  GPU allocated             : 344.72 MB
  GPU reserved              : 1572.00 MB
============================================================

Output files:
  compressed.npz                2293.5 KB
  model.pth                     76942.7 KB
```

Figure 9: Final Results Summarized

reduce compression gains. $K = 16$ is the standard choice in the Han et al. (2016) *Deep Compression* paper that this pipeline is based on.

Losslessness of Huffman: The Huffman encode-decode cycle is mathematically lossless for any symbol sequence. The accuracy of the decoded model is therefore bounded below only by the quantization step, not by the entropy coding step.

Limitations:

- The 90% pruning rate is aggressive. For production use, iterative pruning (gradually increasing sparsity) with more fine-tuning epochs would recover more accuracy.
- The current implementation reconstructs float32 weights at load time. For actual inference speedup, sparse matrix multiplication kernels or INT4 inference engines would be needed.
- Huffman decoding (∼10s) is not suitable for on-device deployment; it is a compression-at-rest technique.

7 Conclusion

The three-stage pipeline achieves a **∼9× compression ratio** on the SmallCIFARNet model for Fruits-360, reducing storage from ∼18.9 MB to ∼2.1 MB, with a **Top-1 accuracy of ∼90.3%** compared to the ∼94.2% baseline — a drop of approximately 3.9 percentage points. The pipeline demonstrates that classical compression techniques (pruning, quantization, entropy coding) remain highly effective for CNN compression, and that their combination is significantly more powerful than any single technique alone.